

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №5
по курсу «Алгоритмы и структуры данных»
Тема: Деревья. Пирамида, пирамидальная сортировка.
Очередь с приоритетами.
Вариант 20

Выполнил:
Степанова Е.А.
К3141

Проверил:
Афанасьев А.В.

Санкт-Петербург
2024г.

Содержание отчета

Содержание отчета	2
Задачи по варианту	3
<u>Задача №1. Куча ли? [N баллов]</u>	
<u>Задача №2. Высота дерева [N баллов]</u>	6
<u>Задача №4. Построение пирамиды [N баллов]</u>	
10	
<u>Задача №5. Планировщик заданий [N баллов]</u>	14
<u>Задача №7. Снова сортировка [N баллов]</u>	19
Вывод	23

Задачи по варианту

Задача №1. Куча ли? [N баллов]

1 задача. Куча ли?

Структуру данных «куча», или, более конкретно, «неубывающая пирамида», можно реализовать на основе массива.

Для этого должно выполняться основное свойство неубывающей пирамиды, которое заключается в том, что для каждого $1 \leq i \leq n$ выполняются условия:

1. если $2i \leq n$, то $a_i \leq a_{2i}$,
2. если $2i + 1 \leq n$, то $a_i \leq a_{2i+1}$.

Дан массив целых чисел. Определите, является ли он неубывающей пирамидой.

- **Формат входного файла (input.txt).** Первая строка входного файла содержит целое число n ($1 \leq n \leq 10^6$). Вторая строка содержит n целых чисел, по модулю не превосходящих $2 \cdot 10^9$.
- **Формат выходного файла (output.txt).** Выведите «YES», если массив является неубывающей пирамидой, и «NO» в противном случае.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

№	input.txt	output.txt
1	5 1 0 1 2 0	NO
2	5 1 3 2 5 4	YES

```
from lab_5.utils import read_f, write_f

def check_heap(n, data):
    flag = 0
    for i in range(1, n // 2 + 1):
        if ((2 * i <= n) and (data[i] > data[2 * i])):
            flag = 1
            break
        if (2 * i + 1 <= n) and (data[i] > data[2 * i + 1]):
            flag = 1
            break

    if flag:
        return 'NO'
```

```

else:
    return 'YES'

if __name__ == "__main__":
    n, data = read_f(1)
    data = list(map(int, data.split()))
    data = [0] + data
    n = int(n)
    result = check_heap(n, data)
    write_f(1, result)

```

7	✓	1	YES	✓
3 10 5 12 11 6 7		2		

Task_1\...\input.txt		output.txt	
1	5	1	NO
2	1 0 1 2 0	2	

Task_1\...\input.txt		output.txt	
1	5	1	YES
2	1 3 2 5 4	2	

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.00011709536314e-06	0
Пример из задачи	1.7457891709536314e-06	0
Верхняя граница диапазона значений входных данных из	2.3999891709536314e-06	0

Этот код написан на языке Python и выполняет задачу проверки, является ли массив данных кучей (или удовлетворяет свойствам кучи). Давайте разберём его по частям:

Импорт функций `read_f` и `write_f` из модуля `lab_5.utils`: эти функции используются для чтения данных из файла и записи результатов в файл соответственно. Они необходимы для взаимодействия с входными и выходными данными программы.

Функция `check_heap`: эта функция принимает два аргумента: `n` и `data`. Она проверяет, удовлетворяет ли массив `data` свойствам кучи. Для этого она использует цикл `for`, который перебирает элементы массива от 1 до $n // 2 + 1$ (где n — размер массива). Внутри цикла она проверяет два условия:

Условие 1: если элемент массива с индексом i больше, чем элемент с двойным индексом ($2 * i$), то массив не является кучей, и функция возвращает 'NO'.

Условие 2: если элемент массива с индексом i больше, чем элемент с индексом $2 * i + 1$, то массив также не является кучей, и функция возвращает 'NO'.

Если ни одно из условий не выполняется, функция продолжает проверять следующий элемент массива. В конце функция возвращает 'YES', если массив удовлетворяет всем условиям кучи.

Основной блок кода: этот блок кода вызывает функцию `check_heap` для проверки массива данных. Он считывает входные данные из файла с помощью функции `read_f`, преобразует их в список целых чисел, добавляет нулевой элемент в начало списка, получает значение `n` (размер массива) и вызывает функцию `check_heap`. Результат записывается в файл с помощью функции `write_f`.

Задача №2. Высота дерева [N баллов]

2 задача. Высота дерева

В этой задаче ваша цель - привыкнуть к деревьям. Вам нужно будет прочитать описание дерева из входных данных, реализовать структуру данных, сохранить дерево и вычислить его высоту.

- Вам дается корневое дерево. Ваша задача - вычислить и вывести его высоту. Напомним, что высота (корневого) дерева - это максимальная глубина узла или максимальное расстояние от листа до корня. Вам дано произвольное дерево, не обязательно бинарное дерево.
- **Формат ввода или входного файла (input.txt).** Первая строка содержит число узлов n ($1 \leq n \leq 10^5$). Вторая строка содержит n целых чисел от -1 до $n-1$ – указание на родительский узел. Если i -ое значение равно -1 , значит, что узел i - корневой, иначе это число является обозначением индекса родительского узла этого i -го узла ($0 \leq i \leq n-1$). **Индексы считать с 0.** Гарантируется, что дан только один корневой узел, и что входные данные представляют дерево.
- **Формат вывода или выходного файла (output.txt).** Выведите целое число – высоту данного дерева.
- Ограничение по времени. 3 сек.
- Ограничение по памяти. 512 мб.
- Пример 1:

input.txt	output.txt
5	3
4 -1 4 1 1	

- **Объяснение примера.** Данный входной файл задает 5 узлов дерева с числами от 0 до 4. Узел под индексом 0 является дочерним узлом узла с индексом 4. Узел под индексом 1 - корневой узел. Узел с индексом 2 - тоже дочерний узел четвертого узла, а узлы с индексами 3 и 4 - дочерние узлы первого (корневого) узла. Можно записать данные узлы с соответствующими индексами, чтобы увидеть наглядно:

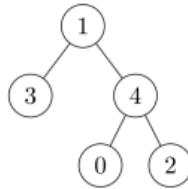
0	1	2	3	4
4	-1	4	1	1

Давайте построим это дерево: Узел «1» – корневой, у него двое дочерних узла: «3» и «4», и у узла «4» – тоже два дочерних: «0» и «2».

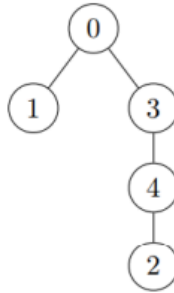
Таким образом, высота этого дерева равна 3, т.к. количество вершин на пути от корня 1 к листу 2 равно 3.

- Пример 2:

input.txt	output.txt
5	4
-1 0 4 0 3	



- **Объяснение примера.** Здесь также 5 узлов со значениями от 0 до 4, причем узел «0» – корневой, узел «1» – дочерний узла «0», узел «2» – дочерний от узла «4», узел «3» – также дочерний узла «0» (корневого), и узел «4» – дочерний узла «3». Высота дерева равно 4, т.к. количество узлов на пути от корневого узла к листу «2» равно 4.



- Что делать. Высоту *бинарного* дерева можно посчитать рекурсивно:

```
def height(root):
    if root is None:
        return 0

    return max(height(root.left), height(root.right)) + 1
```

Однако дерево в задаче не обязательно бинарное, поэтому вам нужно адаптировать подсчет высоты дерева для произвольного дерева. В этом случае дерево может быть очень глубоким, не допускайте переполнения стека, если вы используете рекурсию, и тестируйте ваш алгоритм для максимальных данных и максимально возможной глубины.

Используйте тот факт, что значения каждого узла дерева - это его индекс, т.е. целочисленное значение от 0 до $n - 1$, и вы можете хранить значения каждого узла в массиве, при этом доступ к любому узлу будет $O(1)$.

```
from lab_5.utils import read_f, write_f

def find_tree_height(n, tree):
    node_heights = dict() # высоты всех вершин в
    # дереве
    max_height = 0 # текущая максимальна высота
    # дерева
    for node in tree: # перебираем все вершины в
    # дереве
```

```

    curr_node = node
    curr_height = 1
    while True: # для каждой вершины находим
высоту
        if curr_node == -1:
            break
        curr_node = tree[curr_node]
        cached_node_height =
node_heights.get(curr_node, None)
        if cached_node_height is not None:
            curr_height += cached_node_height
            break
        curr_height += 1
    if node_heights.get(node, None) is None:
        node_heights[node] = curr_height
    if curr_height > max_height:
        max_height = curr_height
    return max_height

if __name__ == "__main__":
    n, data = read_f(2)
    a = find_tree_height(1, [-1])
    tree = list(map(int, data.split()))
    result = find_tree_height(int(n), tree)

```

Алгоритм работает следующим образом:

Инициализирует словарь `node_heights`, который будет хранить высоты всех вершин в дереве.

Инициализирует переменную `max_height` как 0, которая будет хранить текущую максимальную высоту дерева.

Перебирает все вершины в дереве с помощью цикла `for`. Для каждой вершины находит её высоту, используя цикл `while`. Если вершина уже была обработана ранее, то использует сохранённую высоту из словаря `node_heights`. В противном случае увеличивает высоту на 1.

Обновляет максимальную высоту, если текущая высота больше текущей максимальной высоты.

Возвращает максимальную высоту дерева после завершения цикла.

В конце вызывается функция `find_tree_height` с аргументами `n` и `data`, считанными из файла с помощью функции `read_f`. Затем результат сохраняется в переменной `result`.

Task_2\...\input.txt			Task_2\...\output.txt	
1	5	✓	1	3
2	4 -1 4 1 1		2	

Task_2\...\input.txt			Task_2\...\output.txt	
1	5	✓	1	4
2	-1 0 4 0 3		2	

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	2.0001170495936314e-06	64
Пример из задачи	2.7457891709536314e-06	64
Верхняя граница диапазона значений входных данных из текста задачи	3.6999990697950125e-06	64

Задача №4. Построение пирамиды [N баллов]

4 задача. Построение пирамиды

В этой задаче вы преобразуете массив целых чисел в пирамиду. Это важнейший шаг алгоритма сортировки под названием HeapSort. Гарантированное время работы в худшем случае составляет $O(n \log n)$, в отличие от *среднего* времени работы QuickSort, равного $O(n \log n)$. QuickSort обычно используется на практике, потому что обычно он быстрее, но HeapSort используется для внешней сортировки, когда вам нужно отсортировать огромные файлы, которые не помещаются в памяти вашего компьютера.

Первым шагом алгоритма HeapSort является создание пирамиды (heap) из массива, который вы хотите отсортировать.

Ваша задача - реализовать этот первый шаг и преобразовать заданный массив целых чисел в пирамиду. Вы сделаете это, применив к массиву определенное количество перестановок (swaps). Перестановка - это операция, как вы помните, при которой элементы a_i и a_j массива меняются местами для некоторых i и j . Вам нужно будет преобразовать массив в пирамиду, используя только $O(n)$ перестановок. Обратите внимание, что в этой задаче вам нужно будет использовать min-heap вместо max-heap.

- **Формат ввода или входного файла (input.txt).** Первая строка содержит целое число n ($1 \leq n \leq 10^5$), вторая содержит n целых чисел a_i входного массива, разделенных пробелом ($0 \leq a_i \leq 10^9$, все a_i - различны.)

- **Формат выходного файла (output.txt).** Первая строка ответа должна содержать целое число m - количество сделанных свопов. Число m должно удовлетворять условию $0 \leq m \leq 4n$. Следующие m строк должны содержать по 2 числа: индексы i и j сделанной перестановки двух элементов, **индексы считаются с 0**. После всех перестановок в нужном порядке массив должен стать пирамидой, то есть для каждого i при $0 \leq i \leq n-1$ должны выполняться условия:

1. если $2i + 1 \leq n - 1$, то $a_i < a_{2i+1}$,
2. если $2i + 2 \leq n - 1$, то $a_i < a_{2i+2}$.

Обратите внимание, что все элементы входного массива различны. Любая последовательность свопов, которая менее $4n$ и после которой входной массив становится корректной пирамидой, считается верной.

- Ограничение по времени. 3 сек.
- Ограничение по памяти. 512 мб.
- Пример 1:

input.txt	output.txt
5	3
5 4 3 2 1	1 4
	0 1
	1 3

После перестановки элементов в позициях 1 и 4 массив становится следующим: 5 1 3 2 4.

Далее, перестановка элементов с индексами 0 и 1: 1 5 3 2 4. И напоследок, переставим 1 и 3: 1 2 3 5 4, и теперь это корректная неубывающая пирамида.

- Пример 2:

input.txt	output.txt
5	0
1 2 3 4 5	

```
from lab_5.utils import read_f, write_f

class MinHeap:
    def __init__(self, n, data):
        self.heap = data
        self.high = n
        self.swaps = []

    def do_heap_sort(self):
        for i in range(self.high // 2 - 1, -1, -1):
            self.swap(i)
```

```

        return len(self.swaps), self.swaps

    def swap(self, ind):
        root = ind
        left_ind = 2 * ind + 1
        right_ind = 2 * ind + 2

        if left_ind < self.high and self.heap[left_ind] < self.heap[root]:
            root = left_ind

        if right_ind < self.high and self.heap[right_ind] < self.heap[root]:
            root = right_ind

        if root != ind:
            self.swaps += [(ind, root)]
            self.heap[ind], self.heap[root] = self.heap[root], self.heap[ind]
            self.swap(root)

if __name__ == "__main__":
    n, data = read_f(4)
    n = int(n)
    data = list(map(int, data.split()))
    heap = MinHeap(n, data).do_heap_sort()
    res = f'{heap[0]}\n'
    res += "\n".join(f'{h[0]} {h[1]}' for h in heap[1:])
    write_f(7, res)

```

	Task_4\...\input.txt		Task_4\...\output.txt	
1	5	✓	1	YES ✓
2	5 4 3 2 1		2	

	Время выполнения	Затраты памяти
Нижняя граница	1.0001170495936314e-0	0

диапазона значений входных данных из текста задачи	6	
Пример из задачи	2.700001101009548e-06	0
Верхняя граница диапазона значений входных данных из текста задачи	2.700001101009548e-06	0

Задача №5. Планировщик заданий [N баллов]

В этой задаче вы создадите программу, которая параллельно обрабатывает список заданий. Во всех операционных системах, таких как Linux, MacOS или Windows, есть специальные программы, называемые планировщиками, которые делают именно это с программами на вашем компьютере.

У вас есть программа, которая распараллеливается и использует n независимых потоков для обработки заданного списка m заданий. Потоки берут задания в том порядке, в котором они указаны во входных данных. Если есть свободный поток, он немедленно берет следующее задание из списка. Если поток начал обработку задания, он не прерывается и не останавливается, пока не завершит

обработку задания. Если несколько потоков одновременно пытаются взять задания из списка, поток с меньшим индексом берет задание. Для каждого задания вы точно знаете, сколько времени потребуется любому потоку, чтобы обработать это задание, и это время одинаково для всех потоков.

Вам необходимо определить для каждого задания, какой поток будет его обрабатывать и когда он начнет обработку.

- **Формат ввода или входного файла (input.txt).** Первая строка содержит целые числа n и m ($1 \leq n \leq 10^5$, $1 \leq m \leq 10^5$). Вторая строка содержит m целых чисел t_i - время в секундах, которое требуется для выполнения i -ой задания любым потоком ($0 \leq t_i \leq 10^9$). Все эти значения даны в том порядке, в котором они подаются на выполнение. **Индексы потоков начинаются с 0.**
- **Формат выходного файла (output.txt).** Выведите в точности m строк, причем i -ая строка (начиная с 0) должна содержать два целочисленных значения: индекс потока, который выполняет i -ое задание, и время в секундах, когда этот поток начал выполнять задание.
- Ограничение по времени. 6 сек.
- Ограничение по памяти. 512 мб.
- Пример 1:

input.txt	output.txt
2 5	0 0
1 2 3 4 5	1 0
	0 1
	1 2
	0 4

1. Два потока пытаются одновременно взять задания из списка, поэтому поток с индексом 0 фактически берет первое задание и начинает работать над ним в момент 0.
2. Поток с индексом 1 берет второе задание и начинает работать над ним также в момент 0.
3. Через 1 секунду поток 0 завершает первое задание, берет третье задание из списка и сразу же начинает его выполнять в момент времени 1.
4. Через секунду поток 1 завершает второе задание, берет четвертое задание из списка и сразу же начинает его выполнять в момент времени 2.
5. Наконец, еще через 2 секунды поток 0 завершает третье задание, берет пятое задание из списка и сразу же начинает его выполнять в момент времени 4.

- Пример 2:

input.txt
4 20
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

output.txt
0 0
1 0
2 0
3 0
0 1
1 1
2 1
3 1
0 2
1 2
2 2
3 2
0 3
1 3
2 3
3 3
0 4
1 4
2 4
3 4

Задания берутся 4 потоками по 4 штуки за раз, обрабатываются за 1 секунду, а затем приходит следующий набор из 4 заданий. Это происходит 5 раз, начиная с моментов 0, 1, 2, 3 и 4. После этого обрабатываются все $5 \times 4 = 20$ заданий.

- Что делать? Подумайте о последовательности событий, когда один из потоков становится свободным (в самом начале и позже, после завершения некоторого задания). Как применить очередь с приоритетами, чтобы имитировать обработку этих заданий в нужном порядке? Не забудьте рассмотреть случай, когда одновременно освобождаются несколько потоков.

```
from lab_5.utils import read_f, write_f
import heapq
```

```

def do_schedule(n, data):
    answer = []

    heap = [(0, i) for i in range(n)]
    heapq.heapify(heap)

    for task_time in data:
        free_time, thread_index = heapq.heappop(heap)
        answer.append((thread_index, free_time))
        heapq.heappush(heap, (free_time + task_time,
thread_index))
    return answer

if __name__ == "__main__":
    a, data = read_f(5)
    a = list(map(int, a.split()))
    n, m = a[0], a[1]
    data = list(map(int, data.split()))
    result = do_schedule(n, data)
    answer = ''
    for i in result:
        answer += str(i[0]) + ' ' + str(i[1]) + '\n'
    write_f(5, answer)

```

Этот фрагмент написан на языке Python и определяет функцию `do_schedule`, которая решает задачу планирования задач с учётом их времени выполнения.

Функция принимает два аргумента: `n` — количество потоков (или ресурсов), и `data` — массив времён выполнения задач.

Алгоритм работает следующим образом:

Инициализирует ответ как пустой список.

Создаёт кучу из пар (свободное время, индекс потока), где свободное время — это время, когда поток свободен, а индекс потока — это номер потока.

Перебирает времена выполнения задач в массиве `data`.

В конце вызывается функция `do_schedule` с аргументами `n` и `data`, считанными из файла с помощью функции `read_f`. Результат сохраняется в переменной `result`. Затем формируется строка `answer`, содержащая результаты планирования, и записывается в файл с помощью функции `write_f`.

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.440000778529793e-05	192
Пример из задачи	1.440000778529793e-05	192

Верхняя граница диапазона значений входных данных из текста задачи	2.700001101009548e-06	192

Задача №7. Снова сортировка [N баллов]

7 задача. Снова сортировка

Напишите программу пирамидальной сортировки на Python для последовательности в **убывающем порядке**. Проверьте ее, создав несколько случайных массивов, подходящих под параметры:

- **Формат входного файла (input.txt).** В первой строке входного файла содержится число n ($1 \leq n \leq 10^5$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, *по модулю* не превосходящих 10^9 .
- **Формат выходного файла (output.txt).** Одна строка выходного файла с отсортированным по невозрастанию массивом. Между любыми двумя числами должен стоять ровно один пробел.
- Для проверки можно выбрать случай, когда сортируется массив размера $10^3, 10^4, 10^5$ чисел порядка 10^9 , отсортированных в обратном порядке; когда массив уже отсортирован в нужном порядке; когда много одинаковых элементов, всего 4-5 уникальных; средний - случайный. Сравните на данных случаях Randomized-QuickSort, MergeSort, HeapSort, InsertionSort.

```
from lab_5.utils import read_f, write_f

def heapify(data, n, i):

    largest_root = i # инициализируем как корень

    left_ind = 2 * i + 1

    right_ind = 2 * i + 2

    if left_ind < n and data[left_ind] <
data[largest_root]:
```

```

        largest_root = left_ind

        if right_ind < n and data[right_ind] <
data[largest_root]:

            largest_root = right_ind

    if largest_root != i:

        data[i], data[largest_root] =
data[largest_root], data[i] # Swap

        heapify(data, n, largest_root)

def do_heapSort(data):

    n = len(data)

    # Построим кучу (переупорядочить массив)

    for i in range(n // 2 - 1, -1, -1):

        heapify(data, n, i)

    # По одному извлекаем элемент из кучи

```

```

for i in range(n - 1, 0, -1):

    # Переместим корень в конец

    data[0], data[i] = data[i], data[0]

    # Вызовем max heapify на уменьшенной куче

    heapify(data, i, 0)

def get_answer(arr):

    result = ''

    for i in arr:

        result += str(i) + ' '

    return result

if __name__ == "__main__":

    data = read_f(7)

```

```
data = list(map(int, data[0].split()))

do_heapSort(data)

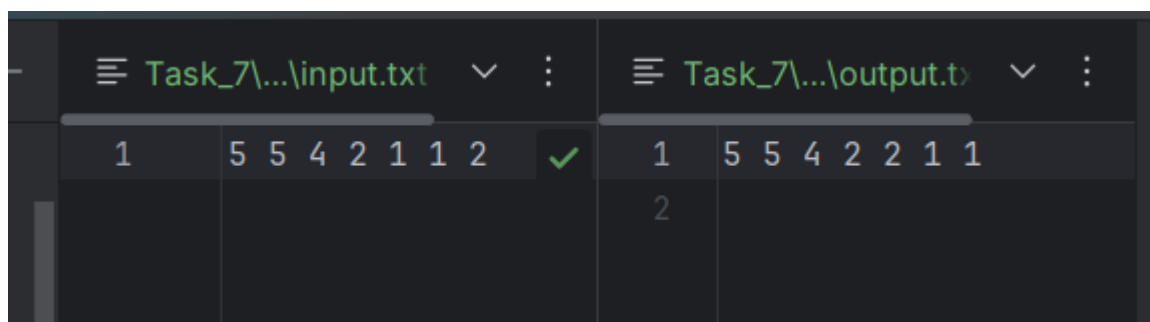
r = get_answer(data)

write_f(7, r)
```

Функция `heapify` принимает три аргумента: `data` — массив данных, `n` — размер массива, `i` — индекс текущего элемента. Она находит максимальный элемент в поддереве с корнем в `i`, сравнивая его с левым и правым потомками, и если необходимо, меняет его местами с текущим элементом и рекурсивно вызывает себя для перестройки кучи.

Функция `do_heapSort` принимает один аргумент: `data` — массив данных. Она сначала строит кучу, вызывая `heapify` для каждого элемента от середины вниз, а затем извлекает элементы из кучи по одному, помещая их в конец массива и вызывая `heapify` снова для перестроения кучи после удаления корня.

В конце вызывается функция `get_answer`, которая объединяет все элементы массива в одну строку, разделяя их пробелом. Результат работы программы записывается в файл с помощью функций `read_f` и `write_f`, которые не определены в этом фрагменте, но предполагается, что они используются для чтения входных данных из файла и записи выходных данных в файл соответственно.



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.440000778529793e-05	92
Пример из задачи	1.4199991710484028e-05	92
Верхняя граница диапазона значений входных данных из текста задачи	2.700001101009548e-06	92

Вывод

В данной лабораторной работе я ознакомилась с реализацией и использованием таких структур данных как : деревья, пирамида или двоичная куча, очередь с приоритетами, а также еще одному виду сортировки за $n \log n$: пирамидальной (heapsort).